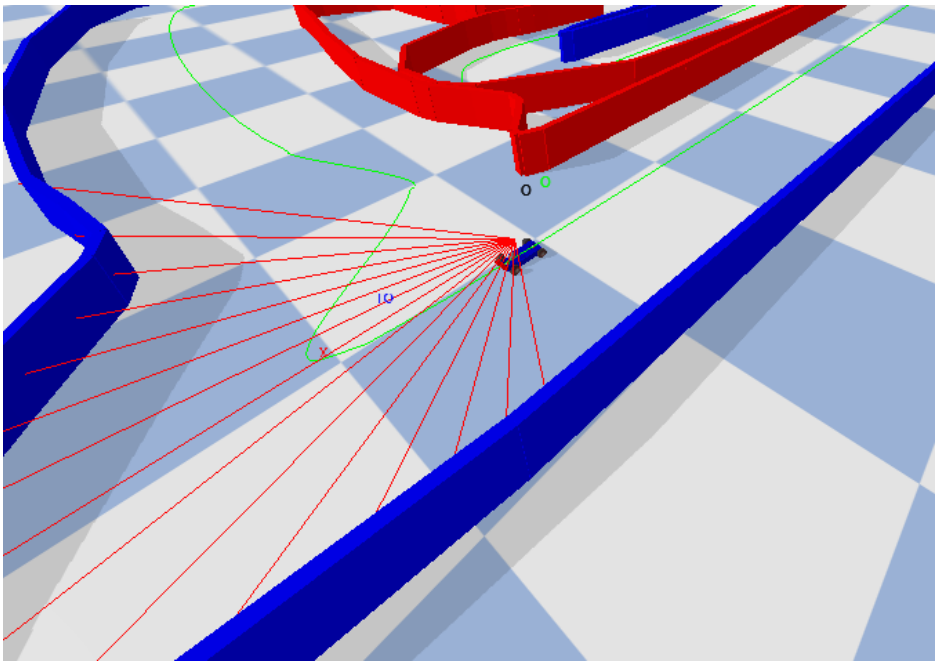
ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE

SEMESTER PROJECT - BIOROB (10ECTS)

FALL SEMESTER 2022

Real-Time Hybrid Control: Combining Reinforcement Learning and Model Predictive Control for Autonomous Systems



Théo HERMANN
SUPERVISION: G. BELLEGARDA, A. IJSPEERT

CONTENTS

Combining MPC with Learning-based Control. Design and Implementation of an RL framework for Autonomous Driving. Benchmark of Different Control Approaches.

Théo Hermann, G. Bellegarda, A. Ijspeert

Abstract - In this work, we introduce a novel hybrid approach for autonomous control that fuses Reinforcement Learning (RL) with Model Predictive Control (MPC). While RL has emerged as a promising technique for controlling complex systems, it can be challenging to apply it in real-world settings and does not always provide performance guarantees. On the other hand, MPC is a well-established method for handling constraints and time-varying systems, but it requires an accurate model and can be computationally intensive. Our proposed hybrid approach combines the strengths of both methods to overcome these limitations and improve performance. We present experimental results on a simulated autonomous racecar to demonstrate the effectiveness of our approach. Our findings demonstrate that the hybrid RL-MPC controller outperforms both an RL and MPC controller individually, and has the potential to drive significant advancements in the field of autonomous control across a wide range of applications.

I. Introduction

Autonomous driving is a complex task that requires the ability to perceive the environment, make decisions, and control the vehicle in real time. Traditional methods for autonomous driving rely on hand-crafted rules and predefined behaviors, which may not be sufficient to handle the uncertainty and variability of the real-world environment. Reinforcement learning (RL) (1) is a promising approach for addressing this challenge, as it allows the control system to learn from data and adapt to changing environments. In this paper, we propose a new approach for autonomous control by combining reinforcement learning with model predictive control (MPC)(2). In recent years, reinforcement learning has emerged as a promising approach for designing controllers for complex systems, it is a powerful tool for learning control policies that can adapt to changing environments and maximize a predefined performance metric, but it can be difficult to transfer learned policies to the real world and generic RL controllers do not have performance guarantees. MPC is a well-established control algorithm but it requires an accurate model of the system and can be computationally demanding.

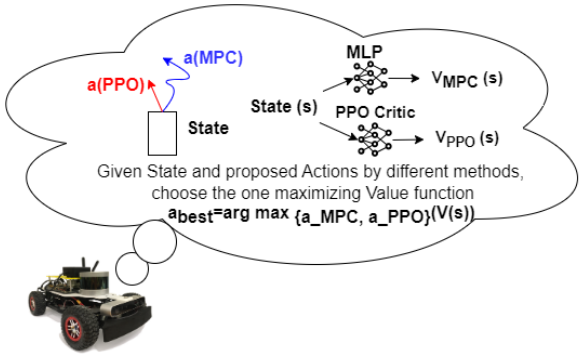


Fig. 1: Agent picking between actions from trajectory optimization or from reinforcement learning based on the value associated with the current state for each control method.

Our method aims to address some of the challenges and limitations of both RL and MPC and to take advantage of their strengths in order to achieve improved performance. One of the main goals of our approach is to enhance the training process by using MPC to guide the RL agent's exploration, resulting in increased sample efficiency. This is particularly important at the beginning of the training process when the RL agent may have limited experience and may need guidance in order to learn effectively. In addition to improving the training process, our method also allows us to guarantee worst-case performance, ensuring that the controller will behave predictably and safely in any situation. This is achieved by switching to MPC when the value of the current state is low for RL and using MPC to ensure that the controller has performance guarantees. This process is described in Figure 1

We propose a hybrid approach in IV that combines RL with MPC to address this challenge. To demonstrate the effectiveness of our approach, we present the results of our experimental study on a simulated environment with a racecar prototype. Our results show that the hybrid RL-MPC controller can perform significantly better than either an RL controller or an MPC controller alone. By combining these two approaches, we aim to take advantage of the strengths of both methods and overcome their limitations. In addition, we discuss the potential future directions for this research.

II. Related Work

A. Combining RL and MPC

There have some previous works that have proposed to combine RL with MPC in order to address the limitations of these methods and improve performance. One of the main motivations for combining RL and MPC is to take advantage of the strengths of both methods. RL is a powerful tool for learning control policies that can adapt to changing environments, while MPC is a well-established method for handling constraints and optimizing performance. By combining these two approaches, it is possible to overcome the limitations of RL and MPC and achieve better performance.

One important work on combining RL and MPC is (3) which proposed an approach for controlling an autonomous car. The authors used RL to learn a control policy in simulation and then used MPC to adapt the policy to the real-world system while taking into account road constraints and performance criteria. The results showed that the hybrid RL-MPC approach was able to outperform either an RL or an MPC controller alone.

In the paper "An Online Training Method for Augmenting MPC with Deep Reinforcement Learning" by Bellegarda and Byl (4), the authors propose a method for combining Model Predictive Control with Reinforcement Learning to increase sample efficiency and guarantee worst-case performance. However, their approach requires simulating actions from both controllers, which makes it impractical for real-time use. In contrast, our method uses a value function approximator to predict the actions and value of the MPC controller, allowing for a more efficient and practical solution for switching between the two controllers.

Our work proposes a new hybrid approach for combining RL and MPC that allows us to switch between these two methods in real time, depending on the value associated with the current state of each method. Specifically, we use RL when the value of the current state is high, indicating that the controller can learn effectively and improve performance. On the other hand, when the value of the current state is low for RL, we switch to MPC to ensure that the controller has performance guarantees and also to guide the training process of RL. An expected result of this approach is an improvement in the training process, as the MPC can guide the RL agent exploration (especially useful at the beginning of the training) resulting in **enhanced sample efficiency** during training. Additionally, our approach allows us to **guarantee worst-case performance**, ensuring that the controller will behave predictably and safely in any situation.

We believe that our approach has the potential to improve autonomous control in a wide range of applications. In the following sections, we describe the details of our method and present the results of our experimental study, which demonstrates the effectiveness of our approach.

B. Other Related Approaches

Hybrid control is a control method that combines different types of controllers, such as PID controllers, state-feedback controllers, and model-based controllers, to achieve better performance than any individual controller could provide. Hybrid control can be particularly useful for systems that exhibit different types of behavior or that operate under varying conditions, as it allows the controller to adapt to different scenarios.

Adaptive control is a control method that adjusts the parameters of the controller based on the current state of the system (5) (6). This can be done using online optimization techniques or by learning from data collected from the system. Adaptive control can be useful for systems that exhibit time-varying or uncertain behavior, as it allows the controller to adapt to changes in the system.

Pure learning-based control is a control method that uses machine learning techniques to learn a control policy from data collected from the system (1) (7). This can be done using supervised learning, unsupervised learning, or reinforcement learning. Learning-based control can be useful for systems that are difficult to model or that exhibit complex or nonlinear behavior, as it allows the controller to learn from experience rather than relying on an accurate model of the system.

Overall, hybrid control, adaptive control, and learning-based control are related to our MPC-RL approach in that they all aim to improve the performance of the controller and adapt to changing conditions. However, these approaches differ in terms of their specific principles and the methods they use to achieve this goal. Hybrid control combines different types of controllers, adaptive control adjusts the parameters of the controller based on the current state of the system, and learning-based control learns a control policy from data.

C. Challenges and limitations

One of the main challenges of combining RL and MPC is determining what should be learned and what should be optimized. While RL algorithms are able to learn policies that maximize the expected reward, they often require a large amount of data and can be sensitive to the choice of reward function. On the other hand, MPC

algorithms can optimize a given cost function using a model of the system, but are limited by the accuracy of the model and can be computationally expensive. Therefore, it is unclear how to effectively mix the two approaches in order to take advantage of their strengths while mitigating their limitations. Additionally, there is the question of which model to use and how to switch between the RL and MPC controllers.

One of the main limitations of RL is the difficulty in transferring the controller learned in simulation to the real world. RL controllers are often learned in simulated environments, where the model of the system is highly accurate and the dynamics are well-controlled. However, when these controllers are deployed in the real world, they may encounter unexpected behavior or variations in the system that were not accounted for in the simulation. This can lead to poor performance or even failure of the controller.

Another challenge is the computational burden of solving the optimization problem for MPC at each time step. MPC relies on solving an optimization problem in order to compute the control action that maximizes a performance metric. This optimization problem can be computationally demanding, especially for high-dimensional systems or systems with complex constraints. This can limit the complexity of the models that can be used and may limit the ability of the controller to adapt to changing conditions.

Despite these challenges, there are many potential applications and areas where combining RL with MPC could be particularly useful. For example, this approach could be applied to autonomous vehicles, where it is important to ensure both safe and efficient operation. It could also be applied to robotic systems, where it is important to adapt to changing environments and maximize performance. In these and other applications, combining RL with MPC has the potential to significantly improve the performance and robustness of the controller.

III. Preliminaries

A. Overview RL and MPC

Reinforcement learning (RL) (1) is a type of machine learning that involves an agent interacting with an environment in order to learn a control policy that maximizes a predefined reward signal. The agent learns by trial and error, making successive actions and receiving feedback in the form of rewards or punishments. RL has been applied to a wide range of control problems, including robotics, game playing (8), and autonomous driving. One of the main advantages of RL is its ability to learn from data and adapt to changing environments, making it a promising approach for handling uncertainty and variability. However, RL can be difficult to apply in the real world, and it does not always provide performance guarantees.

Model Predictive Control (MPC) is a well-established control algorithm that can handle constraints and time-varying systems. MPC works by predicting the future behavior of the system and optimizing a control action that minimizes a predefined performance criterion. MPC requires an accurate model of the system and can be computationally intensive, but it has been widely applied to a variety of control problems, including process control, power systems, and transportation systems. One of the main advantages of MPC is its ability to handle constraints and optimize performance, making it a powerful tool for achieving optimal control.

B. MPC

Model Predictive Control (MPC) is a control method that rely on optimizing a predicted model of the system's behavior over a certain time horizon. It involves iteratively solving an optimization problem at each time step to find the optimal control inputs for the system. The optimization problem for MPC can be formulated as follows: Given a predicted model of the system dynamics, $y = f(x, u)$, where x is the system state and u is the control input, and a cost function, $J = g(x, u)$, find the control input sequence, $u*$, that minimizes the following objective:

$$\begin{aligned} J^* = \min_u \quad & J(x, u) \\ \text{s.t.} \quad & x(k+1) = f(x(k), u(k)) \\ & x(0) = x0 \end{aligned} \tag{1}$$

where k is the current time step and $x0$ is the initial system state.

The solution to this optimization problem provides the optimal control input for the current time step, u(k), as well as a prediction of the future system behavior, x(k+1), x(k+2), ..., x(k+N), where N is the time horizon. This prediction is then used to update the optimization problem and compute the control input for the next time step.

MPC has the advantage of being able to handle constraints on the system and the control input, and it can also take into account the system's future behavior in the optimization process. However, it requires the prediction of the system's model and can be computationally intensive, especially for systems with large state and control spaces.

C. Actor-Critic RL

Reinforcement learning (RL) is a framework for an agent to interact with an environment, modeled as a Markov Decision Process (MDP). An MDP is defined by a tuple (S, A, T, R), where S is the set of states, A is the set of actions available to the agent, $T(s, a, s')$ is the transition function which gives the probability of being in state s, taking action a, and ending up in state s', and $R(s, a, s')$ is the reward function which gives the expected reward for being in state s, taking action a, and ending up in state s'. The goal of the agent is to maximize future rewards by selecting actions that will maximize them.

One type of RL algorithm is the actor-critic algorithm, which consists of two separate neural networks: the actor network, which predicts the actions to take at each state, and the critic network, which estimates the value function of each state. The actor network is updated using the gradient of the value function predicted by the critic network, while the critic network use the Bellman equation to get the value associated $V(s)$ with a state s.

D. Proximal Policy Optimization

Proximal policy optimization (PPO) is a reinforcement learning (RL) algorithm that has been widely used to train agents in complex environments. PPO is an optimization algorithm that aims to maximize the expected return of an RL agent's policy by minimizing the KL-divergence between the new and old policies. The PPO algorithm consists of two main formulas: the PPO objective function and the trust region constraint. The PPO surrogate objective is given by the following equation:

$$L^{CLIP}(\theta) = min(r_t * A_t, clip(r_t, 1-\epsilon, 1+\epsilon) * A_t) \quad (2)$$

where A_t is the advantage at time t, ϵ is a hyperparameter that controls the amount of change allowed in the policy and r_t is the probability ratio between the new and old policies with a_t the current action in state s.

$$r_t(\theta) = \frac{\Pi_{new}(a_t|s_t)}{\Pi_{old}(a_t|s_t)} \quad (3)$$

The trust region constraint is used to ensure that the new policy does not deviate too far from the old policy, and is given by the following equation:

$$KL(\Pi_{old}|\Pi_{new}) \leq \delta \quad (4)$$

where $KL(\Pi_{old}|\Pi_{new})$ is the KL-divergence between the old and new policies, and δ is a hyperparameter that controls the maximum allowed divergence between the policies. By minimizing the PPO objective function subject to the trust region constraint, PPO is able to find policies that are both effective and stable, making it a powerful tool for training RL agents.

PPO is an actor-critic algorithm, the objective function combines the policy surrogate with a value function error term and an entropy bonus to encourage exploration. This full objective function is referred to as $L^{PPO}(\theta)$ and is defined as the expected value of the policy surrogate, L^{CLIP}, minus the hyperparameter-controlled value function error term, L^{VF}, plus the hyperparameter-controlled entropy bonus, S. This can be represented as:

$$L^{PPO}(\theta) = \mathbb{E}[L^{CLIP} - c_1 L^{VF} + c_2 S] \quad (5)$$

where c_1 and c_2 are hyperparameters and L^{VF} is a squared error loss calculated as the difference between the predicted value function and the target value function.

E. Value function

The value function is defined as the expected sum of discounted future rewards, given the current state and action. This can be expressed mathematically as:

$$V(s) = R(s,a) + \gamma V(s') = \sum_{t=0}^{\infty} \gamma^t R_{t+1} \quad (6)$$

where $V(s)$ is the value of state s, s' is the future state, a is the current action, R is the reward function and R_{t+1} is the reward at time $t+1$, and $\gamma = 0.99$ is the discount factor, which determines the importance of future rewards. By using the value function, we are able to estimate the long-term reward of a given state and action, and use this information to guide the learning process of our RL agent. This allows us to learn an optimal policy, which maximizes the expected reward over time.

F. Curriculum Learning

Curriculum learning is a learning paradigm in which an agent is trained on a series of tasks, starting from simple tasks and gradually moving on to more complex tasks. The idea behind this approach is that by starting with simpler tasks, the agent can learn basic skills and build upon them as it progresses through the curriculum. This allows the agent to learn more efficiently and faster, as it can reuse the skills it has already acquired to solve more complex tasks.

One of the main benefits of curriculum learning is that it can help the RL agent to learn faster, as it can leverage previous knowledge to tackle more complex tasks. For example, if the agent is trained on a series of tracks, starting with simpler tracks and gradually moving on to more complex tracks, it can use the skills it has learned on the simpler tracks to navigate the more complex tracks more efficiently. This can help the agent to learn more quickly and effectively, as it does not have to start from scratch each time it encounters a new task. Another benefit of curriculum learning is that it can help to improve the generalization performance of the RL agent, as it is exposed to a wider range of tasks and environments during training. This can be especially useful in situations where the agent needs to operate in a variety of different environments, as it will have had the opportunity to learn a wider range of skills that it can draw upon to adapt to new situations.

Overall, curriculum learning is a powerful tool that can help RL agents to learn faster and more effectively, by leveraging previous knowledge to tackle more complex tasks and improve generalization performance.

G. Car Dynamics

The equations of motion for a robotic system can be written as:

$$D(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) + A(q)^T\lambda = B(q)u + F \quad (7)$$

where q are the generalized coordinates, $D(q)$ is the inertial matrix, $C(q,\dot{q})\dot{q}$ denotes centrifugal and Coriolis forces, $G(q)$ captures potentials (gravity), $A(q)^T\lambda$ are constraint forces (where λ are unknown multipliers a priori), B(q) maps control inputs u into generalized forces, and F contains non- conservative forces such as friction.

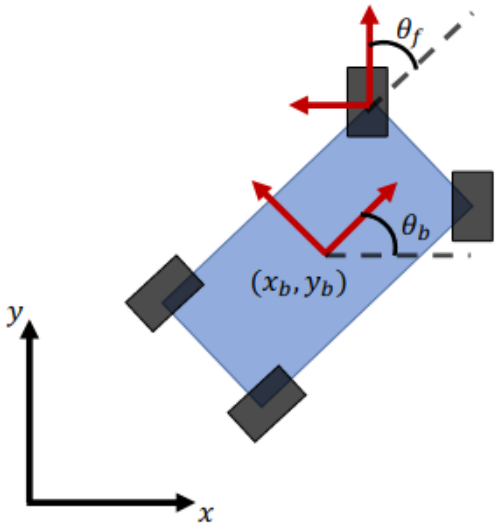


Fig. 2: Car Model used for Trajectory Optimization.

In this work, we specifically consider a simple car model, shown in Figure 2, with $q = [xb, yb, \theta_b, \theta_f]^T$, where (xb, yb) are the center of mass coordinates in the world frame, θ_b is the yaw of the body with respect to the global x-axis, and θ_f is the steering angle of the front wheels. For general wheeled mobile robots, $A(q)^T\lambda$ typically contains constraints ensuring no slip (free rolling in the direction the wheel is pointing) and no skid (no velocity along the wheel's rotation axis perpendicular to the free rolling direction), which come from writing these constraints in Pfaffian form $A(q)\dot{q} = 0$. λ can be explicitly solved for by differentiating $A(q)\dot{q} = 0$ and substituting in $\ddot{q}$ from Equation 7

Remark: In order to prevent skidding during trajectory optimization, a constraint is applied. This means that the optimization process will not find solutions that involve skidding, which could potentially lead to greater rewards (such as skidding into a parking space instead of parallel parking, or aggressively turning to quickly change directions). One alternative to this constraint would be to include friction approximations, but this would require solving a Linear Complementarity Problem at each contact point, which can be computationally expensive. Therefore, we rely on the RL algorithm to use the trajectory optimization as a guide and learn a better policy, including potentially allowing for slip if it is beneficial.

IV. Proposed Approach

A. Methodology

This part presents the high-level information of our proposed approach the next part Implementation IV-B will give more details on the actual execution. Our method is based on the idea of switching between RL and MPC in real time, depending on the value associated with the current state for each method. To implement this approach, we first developed a simulation environment using PyBullet. This **simulation environment** allows us to train and test our controller in a virtual environment on a simulated racecar. After having the **MPC controller** working on the track we started designing the MDP of our **RL-only controller**. Motivated by computational limitations restricting the maximum number of simulations we could run, we had to carefully design the observation and action space of our **Markov Decision Process (MDP)** and spend some time designing precisely the reward function to allow for the car to navigate autonomously on the track.

Afterwards, when the MDP of the RL agent was fixed, we started the **data collection** from the MPC controller. The data we collected is composed of the observation, action taken by MPC and value associated with the state. This data will serve as supervision in the training process of a **Multi-Layer Perceptron** (MLP) used to clone the behavior of the MPC in real time predicting the value and action for a given state so that we can combine both method by choosing which controller to use.

Next, after having both the RL only and MPC controller working, we developed a **hybrid RL-MPC controller** that is able to switch between RL and MPC in real-time. To switch between RL and MPC, we use a value function that estimates the value of the current state for each method. If the value of the current state is high for RL, indicating that the RL controller can learn effectively and improve performance, we use the RL controller. On the other hand, if the value of the current state is low for RL, we switch to the MPC controller to ensure that the controller has performance guarantees and to guide the training process of RL. The different steps of our method are summarized n Figure 3

B. Implementation Hybrid MPC-RL

In this subsection, we present the details of our implementation for combining RL with MPC for autonomous control. We begin by describing the design of the observation and action space, which is crucial for defining the problem and allowing the RL agent to interact with the environment. We then explain our choice of reward function, which guides the learning process of the RL agent and helps it to achieve the desired behavior. Next, we describe our use of supervised learning to train a value function approximator so that we can fuse RL and MPC and the selection of parameters for the neural networks used in our method. Afterthat, we review how

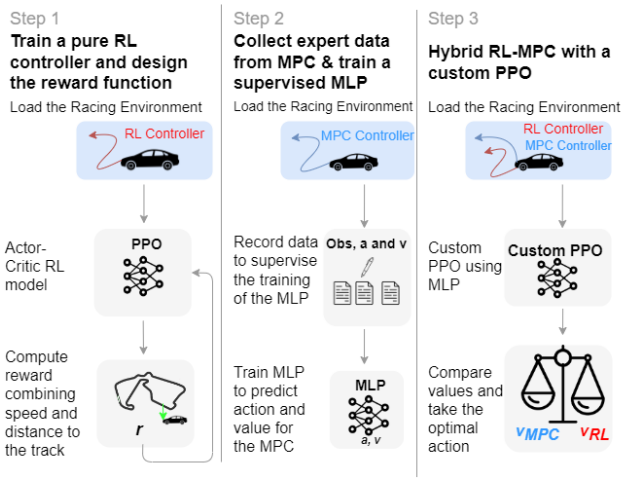


Fig. 3: Diagram Presenting our Approach in 3 step. First the design of a pure RL controller and its reward function, then the data collection process to train the Multi-Layer Perceptron neural network responsible for predicting the value and action taken by the MPC. Finally, how to combine both method in a hybrid MPC-RL controller.

we implement the switching mechanism during training and test time. Finally, we discuss approaches that were not successful or did not yield the desired results, such as previous observation space or reward function. This can help to provide a complete and honest overview of the research process, and be particularly useful for other researchers who may be considering similar approaches in the future.

1) Observation and action space: Our goal is to design an observation and action space that is informative and allow the RL controller to effectively learn and adapt to the dynamic racing environment.

The observation space is :

$$[x_b, y_b, \theta_b, \theta_f, \dot{x_b}, \dot{y_b}, \dot{\theta_b}, \dot{\theta_f}, Li] \tag{8}$$

we used the position (x_b, y_b) and angle (θ_b) of the body of the car on the track, its linear velocity ($\dot{x_b}, \dot{y_b}$), and the distance to the nearest wall or obstacle from each of the 15 lidar sensors (Li_k). This allows the RL model to have a complete understanding of the environment and the car's position within it. We also normalized all observations to ensure that the RL model can learn effectively. The observation dimension is 23 as we have 15 Lidar sensors.

The action space is $[v, \theta_f]$, which is a desired body velocity to be set with velocity control mapped to a differential drive, and desired steering angle to be set with position control.

It is worth noting that the design of the observation and action spaces is a crucial aspect of RL, as it determines the complexity of the learning task and the level of control that the RL model has over the system. In our case, we carefully chose the observations and actions to balance the complexity of the learning task with the level of control required to perform well on the track.

2) Reward function: Our reward function was designed to guide the learning process and help the controller understand the correlation between the lidar information and the optimal trajectory for racing on the track while avoiding collisions. In order to achieve this, we used two measures: the projection of the car's velocity on the track v_b^T and the lateral distance to the track d. By combining these measures into a single function, we were able to control the importance of speed versus trajectory tracking with a single hyperparameter σ. We can see the shape of this function on the 3D Plot from Figure 4

$$R = v_b^T exp(-\frac{d^2}{\sigma^2}) \tag{9}$$

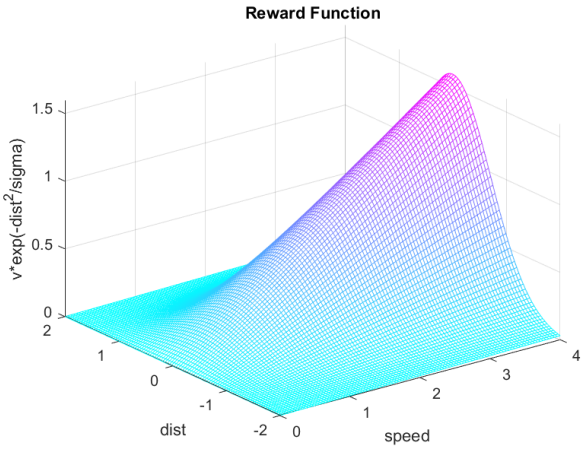


Fig. 4: 3D Plot of our reward function combining the measure of distance to the track and the speed of the car.

This reward function penalizes tracking error using a gaussian function, while maximizing velocity. The benefit of this cost function is that it only contains one hyperparameter σ, which controls the importance of speed versus trajectory tracking.

We also added constant terms to penalize collisions p_{col}, situations where the car was too close to a wall based on a lidar threshold p_{li} and a penalty for staying static p_{st}.

We now discuss the learning curves from Figure 5 for the different reward we tried. First the pink curves called "default" reward just contain the gaussian term combining the speed and distance. Next, the green curve represent the reward with added penalty for collision. Finally in orange is the final reward we decided to use containing all the penalties. We can clearly see the enhanced performance of the agent using the full reward as the reward curve for the training converge to a higher value. The curve counting the number of collision during training is also important to monitor as it gives a good indication of the performances of our model.

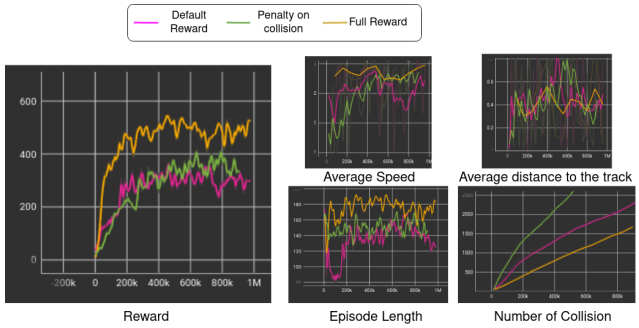


Fig. 5: Plot of the Learning Curves for different rewards

The specific coefficients we used in our case are: $\sigma = 5,\ p_{col} = -10,\ p_{li} = -2,\ p_{st} = -1$

We found that this simple reward function was sufficient for achieving good performance in our simulations even if we decide to only use the RL controller. However, we did try more complex reward functions, but we learned that it is often best to stick with the simplest reward that works. This will be discussed further in the following Alternative approaches and challenges IV-B6 part of the paper.

3) Trajectory Optimization: The trajectory optimization is implemented in Python with CasADi (9), using IPOPT (10) to solve the NLP. Furthermore, our MPC needs a model for the car. In our work we use a **bicycle model** which is a simplified mathematical model commonly used to represent the motion of vehicles, particularly cars. The model is based on the assumption that the vehicle can be treated as a rigid body with two wheels connected by a frame, similar to a bicycle. The model is used to predict the lateral and longitudinal motion of the vehicle, based on the steering angle, velocity, and other factors.

There are two main versions of the bicycle model: the **kinematic** model and the **dynamic model** (presented in III-G). The kinematic model is a simpler version of the model that assumes that the vehicle has a constant velocity and does not consider the forces that act on the vehicle. The dynamic model, on the other hand, takes into account the forces that act on the vehicle, such as the traction and friction forces, and is able to predict the acceleration of the vehicle as well as its velocity. In our method we used the kinematic version of the bicycle model for the MPC due to its simplicity of implementation but we plan on improving this part for the future work. Here is a list of other approach implementing MPC with a bicycle dynamic model (11), (2) and (12). It's also possible to improve the model of the dynamic by **augmenting MPC with Gaussian Processes** as detailed in (13), (14). Finally, (15) propose an approach to fuse both the kinematic and dynamic bicycle model and could be an interesting enhancement for our method.

One of the main advantages of the bicycle model is its simplicity. The model uses a small number of variables and equations, making it relatively easy to understand and implement. This makes it a useful tool for a wide range of applications, including vehicle control, simulation, and analysis.

However, it is important to note that the bicycle model is only a simplified representation of the real-world behavior of a vehicle. The model is based on a number of assumptions and approximations, and is not able to capture all of the complexity and detail of real-world motion. As a result, it is important to use the model appropriately and to be aware of its limitations.

Due to the imposed torque and velocity limits as well as the non-holonomic constraints added to the dynamics, the trajectory optimization will find solutions that are suboptimal to the true best policy. We will see in the following subsection that even this suboptimal trajectory optimization has a large benefit when combined with policies either learned from scratch or with our method, both during training and testing.

4) Value Function Approximator: In order to approximate the value function, we used a **multi-layer perceptron (MLP)** neural network. The MLP was trained in a **supervised learning** framework, using Adam (16) as the optimization algorithm and performing a hyperparameter grid search to find the optimal learning rate. The loss function used was the mean squared error (MSE) between the predicted actions and values, and the supervision data. This allowed us to accurately predict the actions taken by the MPC and the corresponding value of each state, **without solving the optimization problem at each step**. We can see the difference between the predicted and real actions and value in Figure 6 and 7 respectively, in blue is the prediction of the neural network while the dotted black curve is the ground truth during supervised training of the MLP. This not only saves computation time, but also allows us to make online predictions and switch between MPC and reinforcement learning (RL) as needed. By using this value function approximation, we can effectively guide the learning process of RL through the expertise of the MPC. This allows us to incorporate prior knowledge into the RL process, which can improve the **sample efficiency** and speed up the learning process.

Our neural network architecture is the default Multi-Layer Perceptron, consisting of 2 fully connected hidden layers of 128 neurons each, with tanh activation. The policy and value networks each have this same structure. The input dimension is 23 (same as observation dimension) and the output dimension is 3 (2 actions and 1 value).

However, it is important to note that the performance of this value function approximator is highly dependent on the quality of the data used for training, and it can be challenging to collect high-quality data for certain systems or environments (See Challenges IV-B6).

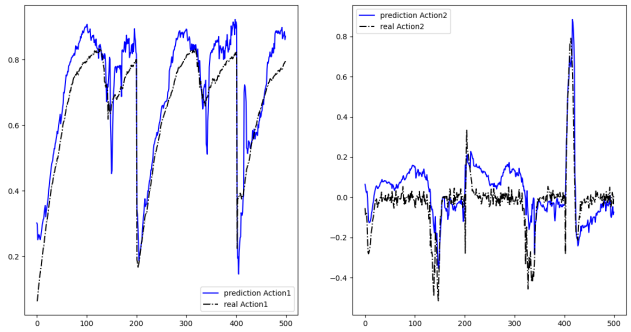


Fig. 6: MLP action prediction.

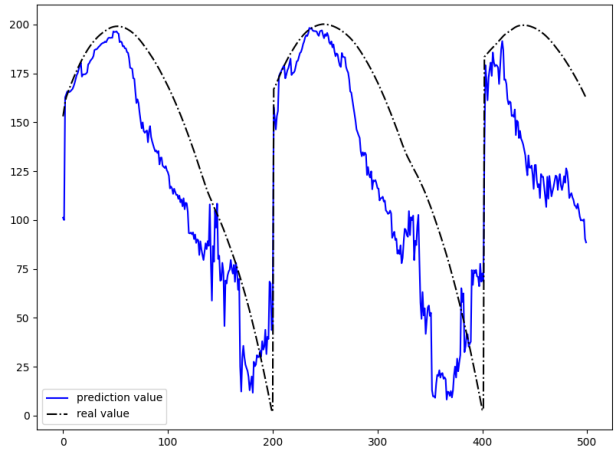


Fig. 7: MLP value prediction.

5) Switching Controllers: At test time, our hybrid RL-MPC controller **switches** between the RL and MPC controllers based on the value function estimated by the MLP neural network as shown in Algorithm 1. If the value of the current state is higher for the RL controller, indicating that it is able to learn effectively and improve performance, the RL controller is used. On the other hand, if the value of the current state is lower for the RL controller, the MPC controller is used to ensure that the controller has **performance guarantees** and to **guide the training** process of RL.

During training, we also switch between the RL and MPC controllers based on the value function estimated by the MLP.

6) Alternative approaches and challenges: We first tested our approach in a empty environment without a track where the goal was to reach a randomly generated target. The observation and reward differed from those used in IV-B1 and IV-B2. In this scenario, the position of the car was not in the observation neither the lidar, only the position of the goal. Concerning the reward it was a simple potential based function giving a reward proportional to the distance to the goal. However, this approach proved to be insufficient when we tried the more complex task of racing on a track. We found that including the position of the car in the observation space was crucial for the RL agent to learn how to navigate the track. However, our attempt to shape the reward based on the lidar values using a complex function depending on multiple hyperparameters inspired from (17) was not successful, even with extensive tuning. We realized that adding this level of complexity to the reward function was hindering the learning process, and ultimately decided not to explicitly use the lidar values in the reward. Instead, we focused on designing a simpler reward function based on the projection of the car's velocity on the track and the lateral distance to the track as described in IV-B2, which proved to be sufficient for guiding the learning process and producing satisfactory results. These challenges highlight the importance of carefully designing the reward function.

Algorithm 1 Switching controllers based on value

Initialize MLP parameters θ,
Initialize PPO parameters ϕ
for $training \quad epoch = 1,2,...$ **do**
  for $timesteps = 1,2,...,T$ **do**
    Observe state s
    Predict value and actions of s for MPC using MLP : $v_{MPC}, a_{MPC} = MLP(s;\theta)$
    Evaluate PPO critic network to get value of s for RL: v_{RL}
    if $v_{RL} > v_{MPC}$ **then**
      Select action a_{RL} using PPO policy network: $a_{RL} = \pi_\theta(s)$
    else
      Select action a_{MPC} predicted by MLP.
    end if
    Take action a and observe reward r and next state s'
  end for
  update PPO Actor-Critic parameters ϕ.
end for

C. Evaluation Metrics

The evaluation metrics used in this study are designed to assess the performance of the hybrid RL-MPC controller and compare it with other control methods.

- Average lap time: measures the time it takes for the controller to complete one lap of the race track.
- Average lateral deviation: (from the center of the track), which reflects the controller's ability to follow the desired path
- Average speed: average speed of the car when racing on a track.
- Success rate: percentage of laps that controller is able to complete without collision.
- Number of collision: Useful to monitor cumulative number of collision at the end of the training process.

Controller/Metrics	Lap Time	Lateral Deviation	Success Rate
RL	17/20/42	0.24/0.25/0.26	100%/100%/83%
MPC	17/22/32	0.22/0.18/0.22	100%/100%/92%
RL Curriculum	-/24/53	-/0.31/0.29	-/87%/75%

TABLE I: Experimental Results

V. Experimental Results

This part presents the results of our experimental study on the effectiveness of our hybrid RL-MPC controller. We conducted a series of experiments on a simulated environment using our small-scale racecar prototype, with the aim of comparing the performance of our hybrid controller with that of an RL controller and an MPC controller alone, using the evaluation metrics described in the previous subsection. Our results can be separated into 2 distinct parts. First, the results using the MDP stated in Proposed Approached IV and using the lidar to navigate around the track. Then, we will also expose result on another empty environment (similar to (4)) without the track and lidar, where we show the benefits of using the Hybrid MPC-RL Controller.

A. Autonomous navigation on tracks

In this part we compare different control methods using the metrics previously stated on 4 tracks of increasing difficulties. The first method is a pure RL controller trained only on the track tested, second we have our MPC controller and finally a RL agent trained with curriculum learning, progressively introducing difficulty in the training process. As we have information on the position of the car around the track the RL-only controller manage to learn pretty fast. We discuss with more details the potential use of our MPC-RL method in this challenging scenario in Discussion V-C. For each controller we have 4 values for the different metrics each corresponding to a track in increasing order of difficulty.

We can see from from results the results presented in the table that the RL algorithm managed to learn really well how to navigate the track autonomously given the reward stated previously. It's important to remark that our implementation of the MPC is currently not perfect for the track we have, as it also struggle to finish the hardest tracks. Future work could focus on implementing a polynomial version of the MPC such as (2). Our experiment with the curriculum learning model did not yield the expected results. One reason for this may be that the position of the car was included in the observation space, which led the car to specialize to a specific track and limited the benefits of the curriculum learning approach. By including the position of the car in the observation space, we may have inadvertently created a strong reward signal for the car to stay on the track, rather than exploring and adapting to new situations. This could have led the car to become overly specialized to the specific track it was trained on, rather than learning more generalizable skills. Overall, our results suggest that the inclusion of the position of the car in the observation space may have limited the benefits of the curriculum learning model and contributed to its lower performance compared to the other models. In future work, it will be important to carefully consider the design of the observation space and to ensure that it does not create unintended reward signals or limitations on the learning process.

B. MPC-RL on simple environment

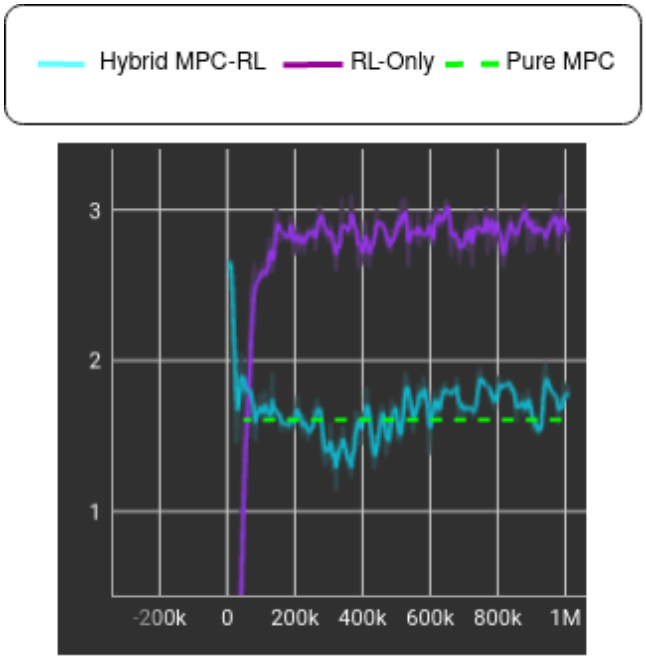


Fig. 8: Comparision of the training reward for the Hybrid MPC-RL and pure RL controllers. Improvement of the Sample Efficiency for MPC-RL but limitated convergence equivalent to pure MPC. Results discussed below.

In this scenario, we considered a simple Markov Decision Process (MDP) where the state space did not include the position of the car or the lidar readings. Instead, the state space only included the goal position, which was provided as input to the MDP. The reward function for this MDP was based on a potential field, which was proportional to the distance between the car and the goal position. This reward function provided a strong incentive for the car to approach the goal. We tested our approach in this simple environment because the given implementation of the Model Predictive Control (MPC) algorithm was able to work effectively in this setting. However, future work should focus on adapting the MPC algorithm to work in more complex environments, such as tracks with obstacles or other challenges. This will allow us to test our proposed approach in more realistic and challenging scenarios, and to better understand its performance and limitations.

Our method for combining Model Predictive Control with Reinforcement Learning demonstrated improved

sample efficiency, as shown by the curve in our results (see Figure 8). This curve shows that the MPC-RL method was able to achieve a higher reward at the beginning of the learning process, compared to the pure RL method. This is likely due to the expert guidance provided by the MPC, which helped to direct the learning process towards more successful actions and outcomes.

However, one limitation of our method is that it tended to converge to a lower reward value compared to the pure RL method. This may be because the MPC constraints and guidance limited the freedom of the RL to explore riskier or more unconventional trajectories, which may have had the potential to lead to higher reward values.

C. Discussion

One of the challenges we faced in this study was the limited computational resources available to us, which made it difficult to run a large number of simulations in order to average the results and reduce the impact of randomness. To address this challenge, we recommend using a cluster or other parallelization techniques to speed up the simulation process and allow for more efficient use of computational resources.

Another challenge we identified was the quality of the data used to supervise the training of the MLP in the Hybrid MPC-RL controller. The performance of the Hybrid MPC-RL controller was highly dependent on the quality of this data, and any errors or biases in the data could significantly impact the performance of the controller. As we said earlier, our current implementation of the Model Predictive Control is not able to track a curve for the track. This is a significant limitation, as the ability to follow a curved track is a key requirement for our application. To address this limitation, we plan to upgrade our MPC implementation to a polynomial MPC (2), which is able to track a curve with greater accuracy and precision. Additionally, we plan to augment our MPC implementation with Gaussian Processes (GPs)(13) (14) in order to improve its ability to handle uncertain or noisy inputs. GPs are a powerful tool for modeling and predicting the behavior of complex systems, and they have been successfully applied in a variety of fields including autonomous driving. By incorporating GPs into our MPC, we hope to improve the robustness and adaptability of our system, and to better handle the unpredictable and dynamic nature of real-world environments. Overall, we believe that the combination of a polynomial MPC and other enhancements to our system will help us to achieve improved performance and reliability.

In our experiments, we found that using curriculum learning, where the agent begins learning on simpler tracks and gradually progresses to more complex tracks, did not have such a great impact on the learning performance. Even if the agent is supposed to build upon its previous knowledge and experiences on the simpler tracks, allowing it to more easily adapt to the more complex tracks; here our RL agent has access to its position in the observation which allowed it to more easily learn to navigate on a specific track, as it had a clear understanding of its location and surroundings. Overall, the use of curriculum learning could be even more beneficial if we are not in a scenario including positional information in the observation which in our case proved to be crucial in enabling the RL agent to quickly and effectively learn to navigate a race track. Finally, we noted that the inclusion of the position of the car in the observation space had a significant impact on the learning process, and could lead to specialization to a specific track. While this may be beneficial in some cases, it could also limit the adaptability and generalizability of the Hybrid MPC-RL controller. In future work, it will be important to carefully consider the tradeoffs involved in including the position of the car in the observation space, and to identify approaches that can balance the benefits of specialization with the need for adaptability and generalization.

VI. Experimental Setup

In this part, we describe the details of the experimental setup we used to evaluate the proposed approach. This includes information about the software used, the simulation environment, the test cases, and any other relevant details.

A. Simulation Environment

We use a combination of OpenAi Gym (18) to represent the MDP and PyBullet (19) as the physics engine for training and simulation purposes. We additionally use the Stable Baselines (20) implementation of PPO (which optimizes as discussed in III-C with the default hyperparameters. The Gym environment we use is similar to the standard HumanoidFlagRun environments, but the humanoid is replaced with the Bullet MIT racecar. Example snapshots from the environment are shown in Figure 10. The track and walls are genreated using custom scripts based on real F1 Grand-Prix schematics.

There are several other Python-based simulators that could potentially be used in place of PyBullet for our reinforcement learning environment. One option is the Gazebo simulator (21), which is designed specifically for robotics applications and provides a wide range of models and features for building complex simulations. Another one is Isaac Gym(22) which is a robotics environment for reinforcement learning that has been developed by NVIDIA. It is based on the popular OpenAI Gym framework, but has been specifically designed for use with robotics simulations. One of the main advantages of Isaac Gym is its compatibility with the Isaac Simulator, which is a powerful and realistic robotics simulation platform. This makes it easy to create and test reinforcement learning algorithms for a wide range of robotics applications. Another benefit of Isaac Gym is its support for multi-robot scenarios, which is important for many real-world robotics applications. This

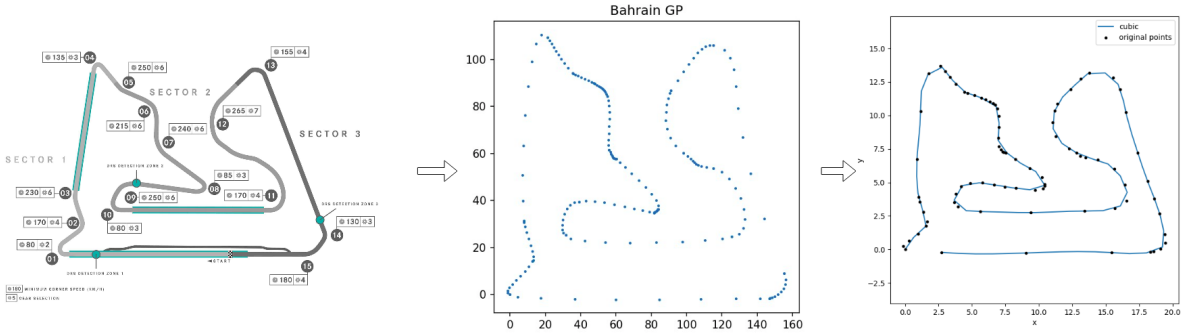


Fig. 9: Illustration of the successive steps to recreate real F1 Grand-Prix Tracks

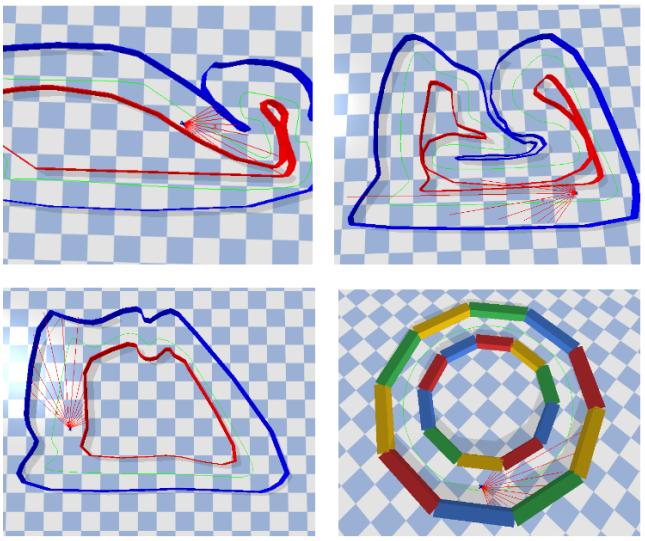


Fig. 10: Snapshots of our environment showing examples of the tracks implemented

sensors were generated using PyBullet and were used to gather information about the environment surrounding the car. This information was then added to the observation space to help the car navigate the track and avoid collisions. The lidar sensors were an integral part of the car model, as they provided important information about the environment that the RL algorithm used to make decisions about the actions to take.

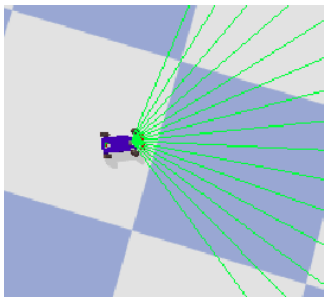


Fig. 11: Simulated Lidar sensors in PyBullet

allows users to train and evaluate reinforcement learning algorithms for coordinating the actions of multiple robots.

B. Model of the car

In the experimental setup of our paper, we utilized Python with the PyBullet library to simulate our racecar inspired by the F1/10 model.

We use the kinematic bicycle model, which is a simplified model of a car that is commonly used in control and robotics. It is based on the assumption that the car is a bicycle with a steerable front wheel and a fixed rear wheel. The model represents the car as a single point mass with a fixed length L, which represents the distance between the front and rear wheels. The kinematic bicycle model is useful for control and planning because it is simple and easy to implement, yet it captures the essential dynamics of a car. It is widely used in robotics and autonomous vehicle research as a benchmark for testing new control and planning algorithms.

Our model of the car also included 15 lidar sensors with a field of view of 120 degrees (see Fig. 11). These

C. Tracks

In this part we describe the process that we implemented in our custom scripts to build F1 racing tracks in our environment as described in Figure 9. To generate racing tracks for our environment, we started by finding real F1 GP racetrack schematics online. Using the software GIMP (GNU Image Manipulation Program) which is a free and open-source raster graphics editor, we drew the path of the track and exported it as an SVG file. Using the python library "svgpathtools", we wrote custom scripts to convert the generated path into an interpolated curve using cubic interpolation. Cubic interpolation is a method of estimating the value of a continuous function at any given point based on the values of the function at surrounding points. It involves fitting a curve (our track) to a set of data points (path drawn on GIMP) using a polynomial equation of degree three (hence the term "cubic"). This curve is then used to interpolate the value of the function at any desired point within the range of the data points.We also used a custom python script to generate an URDF file for the interior and exterior walls of

the track, which was then easily loaded into our Pybullet environment. This allowed us to have realistic and varied tracks for our RL agent to navigate.

VII. Conclusion and Future Work

In conclusion, our proposed approach for combining reinforcement learning with model predictive control has demonstrated its effectiveness in autonomous control through experimental results on a simulated small-scale racecar prototype. Our method allows us to switch between RL and MPC in real-time, depending on the value of the current state for each method. By combining these two approaches, we are able to take advantage of the strengths of both methods and overcome their limitations, resulting in significantly improved performance compared to using either method alone. Additionally, our approach has the potential to be a powerful tool for autonomous control in a wide range of applications. However, there are still limitations and challenges to be addressed in this field, including improving the transferability of learned policies to the real world and developing efficient algorithms for online model adaptation. Further research is needed to address these issues and to advance the field of combining RL with MPC for autonomous control.

Future work could focus on addressing some of the limitations and challenges mentioned in this paper, as well as exploring potential applications of this approach in other domains. Furthermore, one potential direction for future work is to extend our hybrid RL-MPC approach to handle more complex and dynamic environments. This could involve incorporating more sophisticated models and learning algorithms that are able to capture the dynamics of the system more accurately. Another possibility is to investigate the use of our hybrid approach in a real-world setting, by implementing it on a physical race car prototype or another autonomous system. Finally, it would be interesting to explore the use of our approach in combination with other control methods, such as adaptive or hybrid control, in order to further improve performance and robustness.

We want to thank the Biorobotics (BioRob) Laboratory which supported the research project. We could acquire knowledge and material to implement our method, special acknowledgment to G. Bellegarda and A. Ijspeert for supervising this project.

REFERENCES

[1] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. Cambridge: MIT press, 1998.

[2] J. Petr Listov, "Polympc: An efficient and extensible tool for real-time nonlinear model predictive tracking and path following for fast mechatronic systems," *Optimal Control Applications and Methods*, 2017.

[3] J. Lubars and X. e. a. Wu, "Combining reinforcement learning with model predictive control for on-ramp merging," *arXiv:2011.08484*, 2020.

[4] G. Bellegarda and K. Byl, "An online training method for augmenting mpc with deep reinforcement learning," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.

[5] K. J. Astrom and B. Wittenmark, *Adaptive control*. London: Prentice Hall, 1995.

[6] K. S. Narendra and A. M. Annaswamy, *Stable adaptive systems*. Prentice-Hall, Inc., 1989.

[7] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. Cambridge: MIT press, 2016.

[8] D. Silver and D. e. a. Hassabis, "Mastering the game of go with deep neural networks and tree search," *Nature*, 2016.

[9] J. Andersson, J. Gillis, G. Horn, J. Rawlings, and M. Diehl, "Casadi - a software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, 2019.

[10] A. Wächter and L. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical programming*, 2006.

[11] J. Kabzan and R. e. a. Siegwart, "Amz driverless: The full autonomous racing system," *arXiv preprint arXiv:1905.05150*, 2019.

[12] J. Kong and G. e. a. Schildbach, "Kinematic and dynamic vehicle models for autonomous driving control design." IEEE, 2015.

[13] L. Hewing, J. Kabzan, and M. Zeilinger, "Cautious model predictive control using gaussian process regression," *arXiv preprint arXiv:1705.10702*, 2017.

[14] l. Hewing, A. Liinger, and M. Zeilinger, "Cautious nmpc with gaussian process dynamics for autonomous miniature race cars," *arXiv preprint arXiv:1711.06586*, 2017.

[15] G. Bellegarda and Q. Nguyen, "Dynamic vehicle drifting with nonlinear mpc and a fused kinematic-dynamic bicycle model," *IEEE Control Systems Letters*, 2021.

[16] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[17] L. Manuelli and P. Florence, "Reinforcement learning for autonomous driving obstacle avoidance using lidar," *Massachusetts Institute of Technology*, 2019.

[18] OpenAI, "Openai gym," https://gym.openai.com/, 2015.

[19] E. Coumans, "Pybullet: Python bindings for the bullet physics sdk," https://pybullet.org/, 2017.

[20] J. Pérolat, "Stable baselines," https://stable-baselines.readthedocs.io/, 2018.

[21] G. R. Simulator, "Gazebo: A 3d dynamics simulator for autonomous robots," http://gazebosim.org/, 2019.

[22] I. Gym, "Isaac gym: A robotics environment for reinforcement learning," https://developer.nvidia.com/isaac-gym, 2019.